UNIT - 2

ASSIGNMENT – 2

WEB TECHNOLOGIES

SWETHA V

2117230070164

B TECH AI&DS -C

Dynamic DOM Manipulation Website with Progress Tracking

1. Introduction

Web applications today rely heavily on interactive user interfaces. One of the most important technologies that enables dynamic interaction on web pages is the **Document Object Model (DOM)**. DOM manipulation allows developers to dynamically modify the structure, content, and style of a web page using JavaScript.

This project focuses on building a **Dynamic DOM Manipulation Website** that allows users to add, remove, and manage tasks interactively. The application also includes visual feedback through a progress bar and animations, improving the overall user experience.

The project demonstrates how HTML, CSS, and JavaScript can be combined to create a responsive and interactive web application without requiring complex frameworks.

2. Objectives

The main objectives of this project are:

- To understand and implement **DOM manipulation using JavaScript**.
- To dynamically **add and remove HTML elements**.
- To implement **interactive user actions** such as clicking buttons and checking tasks.
- To create a **task progress tracking system**.
- To enhance user experience with **animations and visual effects**.
- To develop a **simple and attractive task management interface**.

3. Technologies Used

HTML

HTML (HyperText Markup Language) is used to create the structure of the website. It defines elements such as headings, input fields, buttons, lists, and sections.

CSS

CSS (Cascading Style Sheets) is used to style the website. It controls the layout, colors, animations, and responsive design of the interface.

JavaScript

JavaScript is used to implement the dynamic functionality of the website. It allows the program to manipulate the DOM, handle user interactions, and update the progress bar.

4. System Architecture

The project follows a simple **client-side architecture**.

User Interaction
↓
HTML Interface
↓
JavaScript DOM Manipulation
↓
Dynamic UI Updates

When the user performs an action such as adding a task or marking it complete, JavaScript updates the DOM and reflects the changes instantly on the web page.

5. Features of the System

The main features of the project include:

Add Task

Users can enter a task in the input field and click the **Add Task** button. The task will dynamically appear in the task list.

Delete Task

Each task has a delete button that allows the user to remove the task from the list.

Remove All Tasks

The **Remove All** button clears all tasks from the list at once.

Task Completion

Each task contains a checkbox that allows users to mark it as completed. Completed tasks appear with a strike-through effect.

Progress Bar

A progress bar visually represents how many tasks have been completed. As tasks are completed, the progress bar fills accordingly.

Theme Switching

Users can switch between **light mode and dark mode** to improve readability and appearance.

Animations and Effects

The project includes modern UI effects such as:

- Task fade-in animation
- Button hover effects
- Animated background
- Smooth delete animation
- Progress bar glow effect

6.Program:

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Smart Kanban Board</title>
<link rel="stylesheet" href="s.css">
</head>
<body>
<header>
<h1>Smart Kanban Task Board</h1>
<button id="themeBtn">Toggle Theme</button>
</header>
<div class="add-task">
<input type="text" id="taskInput" placeholder="Enter new task">
<button onclick="addTask()">Add Task</button>
</div>
<div class="board">
<div class="column" id="todo">
<h2>Todo</h2>
<div class="task-list" ondrop="drop(event)"
ondragover="allowDrop(event)"></div>
</div>
<div class="column" id="doing">
```

```
<h2>Doing</h2>
<div class="task-list" ondrop="drop(event)"
ondragover="allowDrop(event)"></div>
</div>
<div class="column" id="done">
<h2>Done</h2>
<div class="task-list" ondrop="drop(event)"
ondragover="allowDrop(event)"></div>
</div>
</div>
<script src="s.js"></script>
</body>
</html>
```

CSS

```
body{
font-family:Arial;
margin:0;
background:#f4f4f4;
transition:0.3s;
}
header{
display:flex;
justify-content:space-between;
align-items:center;
padding:20px;
background:#333;
color:white;
```

```
}
.add-task{
text-align:center;
margin:20px;}
input{
padding:10px;
width:250px;}
button{
padding:10px 15px;
border:none;
background:#007BFF;
color:white;
cursor:pointer;
border-radius:5px;}
.board{
display:flex;
justify-content:space-around;
padding:20px;
gap:20px;}
.column{
flex:1;
background:#e3e3e3;
padding:15px;
border-radius:10px;
min-height:300px;
}
```

```
.column h2{
text-align:center;
}
.task{
background:white;
padding:10px;
margin:10px 0;
border-radius:6px;
cursor:grab;
display:flex;
justify-content:space-between;
align-items:center;
}
.delete{
background:red;
padding:5px 8px;
border-radius:4px;
}
.dark{
background:#121212;
color:white;
}
.dark .column{
background:#333;
}
.dark .task{
```

```
background:#444;
}
```

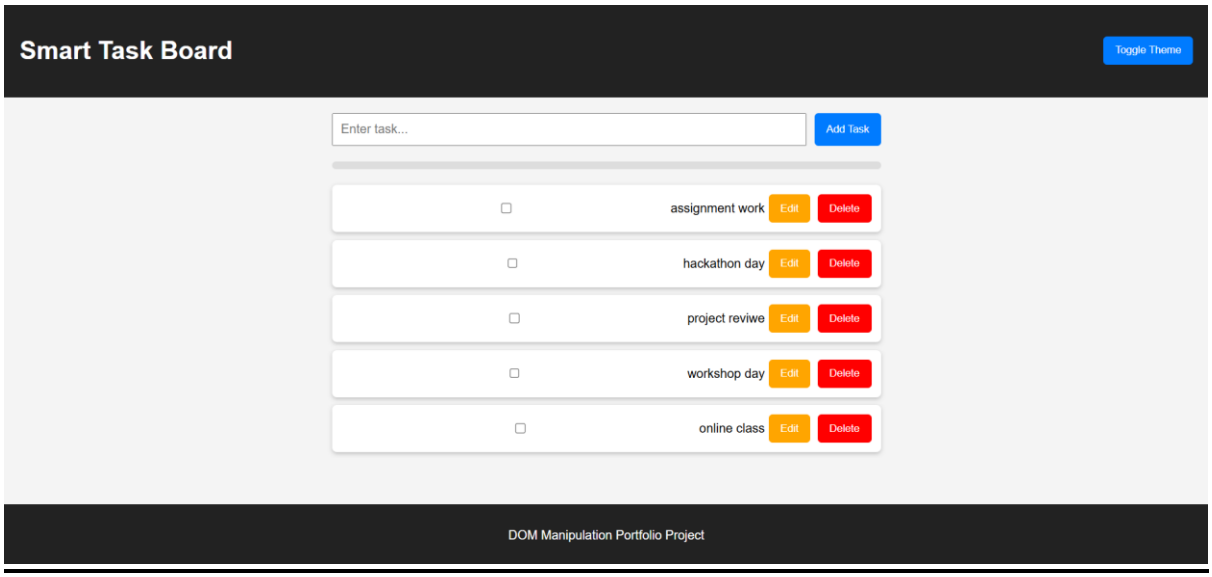
JAVASCRIPT

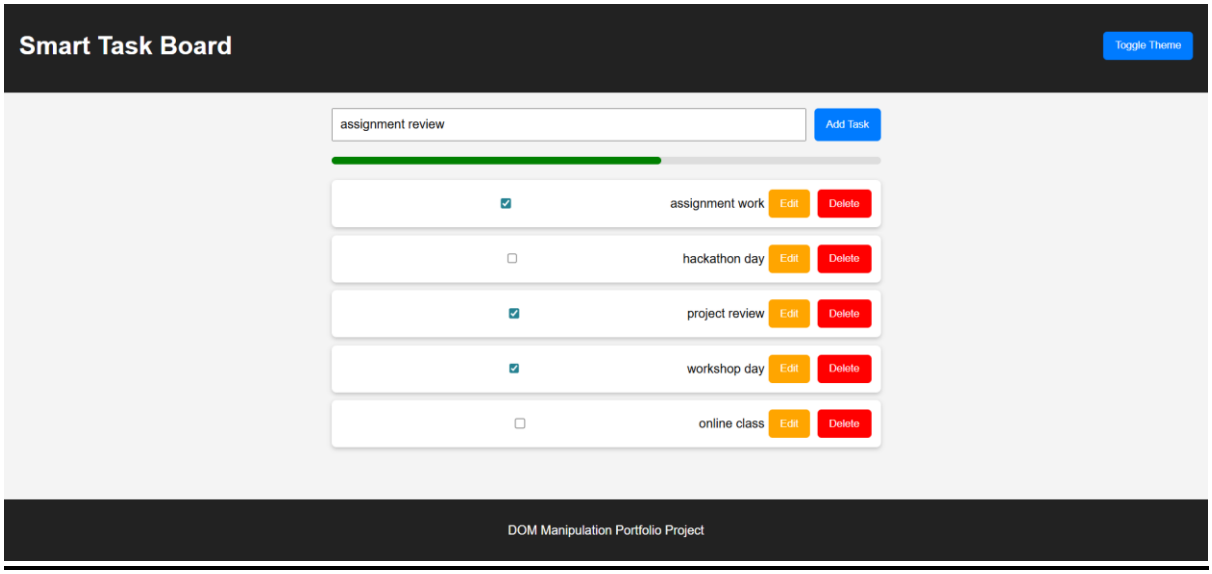
```
let taskId=0;
function addTask(){
const input=document.getElementById("taskInput");
if(input.value==="") return;
const task=document.createElement("div");
task.className="task";
task.id="task"+taskId++;
task.draggable=true;
task.innerHTML=`
<span>${input.value}</span>
<button class="delete">X</button>
`;
task.addEventListener("dragstart",drag);
task.querySelector(".delete").onclick=()=>{
task.remove();
saveBoard();
};
document.querySelector("#todo .task-list").appendChild(task);
input.value="";
saveBoard();
}
function allowDrop(ev){
ev.preventDefault();
```

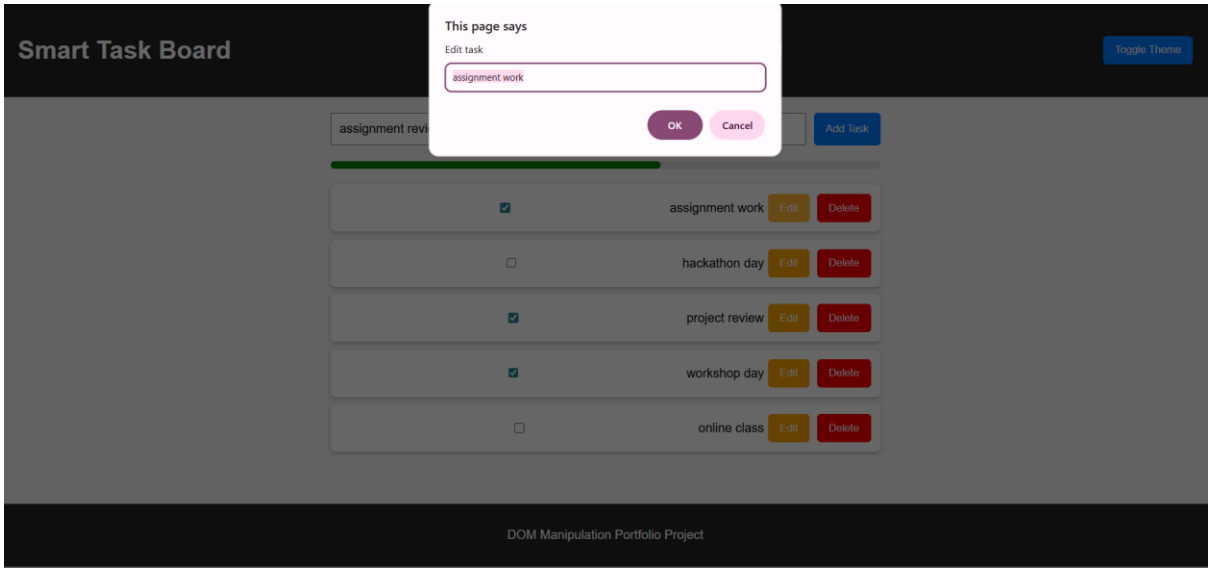
```
}
function drag(ev){
ev.dataTransfer.setData("text",ev.target.id);
}
function drop(ev){
ev.preventDefault();
const id=ev.dataTransfer.getData("text");
const task=document.getElementById(id);
ev.target.closest(".task-list").appendChild(task);
saveBoard();
}
document.getElementById("themeBtn").onclick=()=>{
document.body.classList.toggle("dark");
};
function saveBoard(){
localStorage.setItem("kanban",document.querySelector(".board").innerHTML);
}
window.onload=()=>{
const data=localStorage.getItem("kanban");
if(data){
document.querySelector(".board").innerHTML=data;
document.querySelectorAll(".task").forEach(task=>{
task.addEventListener("dragstart",drag);
});
document.querySelectorAll(".delete").forEach(btn=>{
btn.onclick=()=>{
```

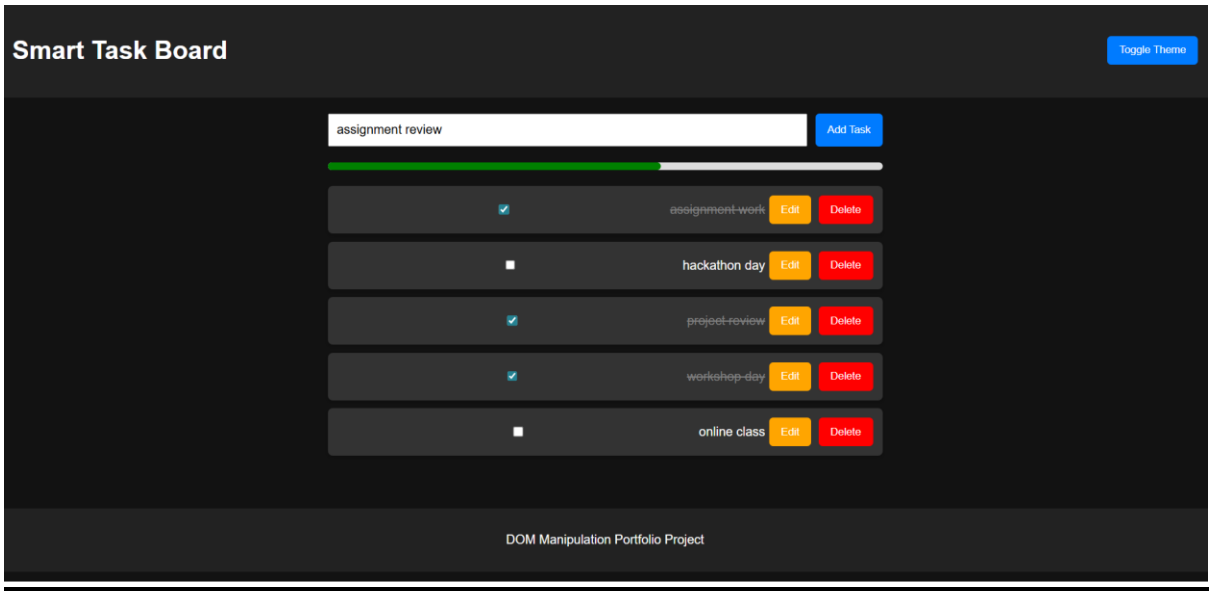
```
btn.parentElement.remove();
saveBoard();
};
});
}
};
```

7.Output









8. Working of the System

1. The user enters a task in the input box.
2. When the **Add Task** button is clicked, JavaScript creates a new <li> element dynamically.
3. The task is added to the task list using DOM manipulation.
4. Each task includes a checkbox and delete button.
5. When the checkbox is selected, the task is marked as completed.

6. The progress bar automatically updates based on completed tasks.
7. When the delete button is clicked, the task is removed from the list.
8. The **Remove All** button clears all tasks from the list.

9. Advantages

- Simple and user-friendly interface
- Interactive and responsive design
- Demonstrates core JavaScript concepts
- Easy to expand with additional features
- No external libraries required

10. Limitations

- Tasks are not stored permanently (unless LocalStorage is added)
- Designed for small-scale usage
- No user authentication or database integration

11. Future Enhancements

This project can be improved with several additional features such as:

- Saving tasks using **LocalStorage**
- Adding **task categories**
- Implementing **drag-and-drop task management**
- Adding **task deadlines and reminders**
- Converting the project into a **full-stack application**
- Integrating with **databases like MySQL**

12. Conclusion

The Dynamic DOM Manipulation Website successfully demonstrates how JavaScript can be used to create an interactive and responsive web application. By dynamically modifying HTML elements and updating styles based on user actions, the project provides a clear understanding of DOM manipulation techniques.

The inclusion of a progress bar, animations, and theme switching improves the overall user experience. This project serves as a strong foundation for learning advanced frontend development concepts.